

08 — Infrastructure Architecture

Purpose

The purpose of this document is to define the infrastructure architecture of Mosaica.

This module establishes the technical foundation required to run, deploy and maintain the Mosaica application.

The Infrastructure Architecture defines how the application is hosted, operated and supported while preserving:

- scalability,
 - reliability,
 - maintainability,
 - security,
 - operational simplicity.
-

Infrastructure Architecture Scope

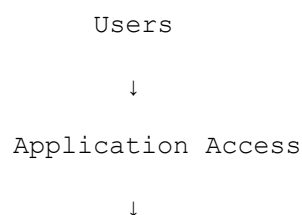
This module defines:

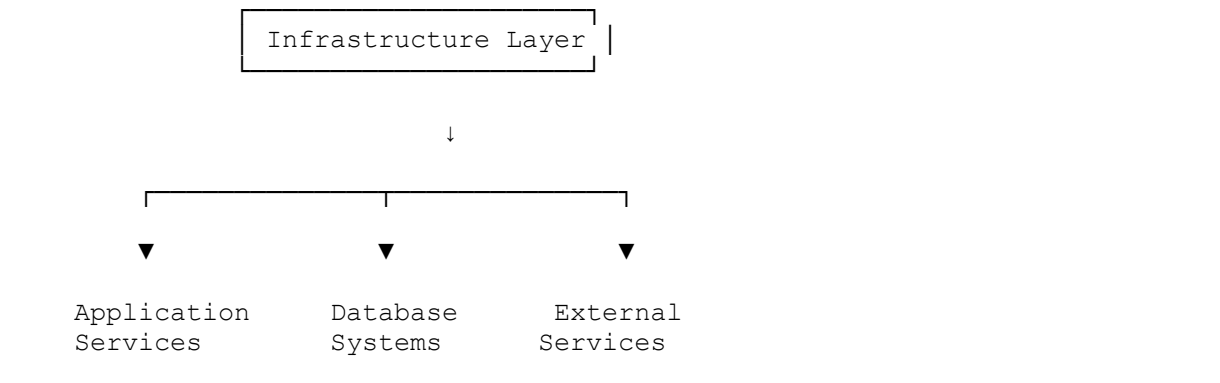
- infrastructure principles,
- deployment structure,
- environment separation,
- external service integration,
- monitoring approach,
- reliability requirements.

The purpose is not to select specific vendors or technologies at this stage, but to establish architectural requirements.

Infrastructure Architecture Position

Infrastructure supports all application layers.





Infrastructure Responsibilities

Application Hosting

Responsible for running:

- frontend applications,
- backend services,
- API infrastructure.

Database Hosting

Responsible for:

- persistent data storage,
- database availability,
- backup support.

Network Infrastructure

Responsible for:

- secure communication,
- service connectivity,
- controlled access.

Operational Infrastructure

Responsible for:

- monitoring,
 - logging,
 - deployment processes,
 - system maintenance.
-

Infrastructure Architecture Principles

Principle 1 — Simplicity First

Infrastructure decisions shall prioritize maintainability and operational simplicity.

Unnecessary infrastructure complexity shall be avoided.

Principle 2 — Scalability When Needed

The architecture shall support future growth without requiring complete redesign.

Scaling decisions shall follow actual system requirements.

Principle 3 — Environment Separation

Development, testing and production environments shall remain separated.

Principle 4 — Reliability By Design

Infrastructure shall support stable operation through:

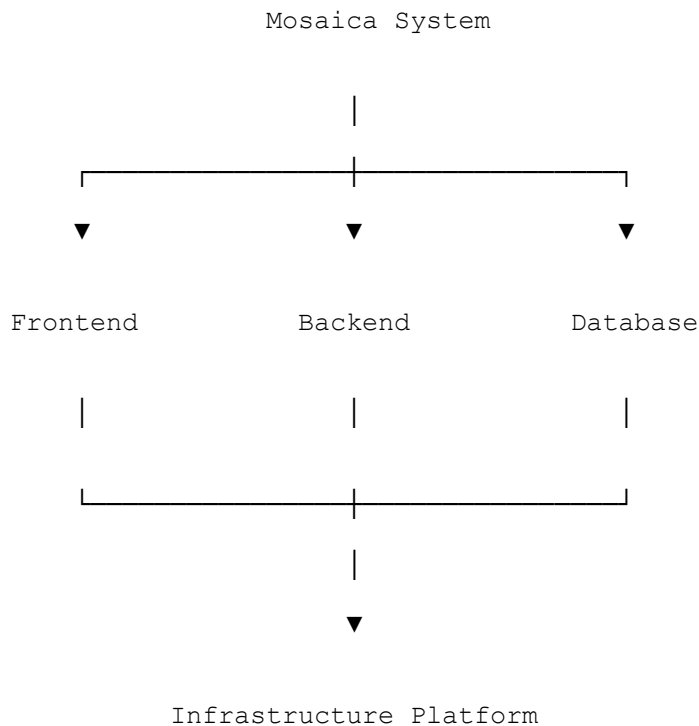
- monitoring,
 - backups,
 - recovery strategies.
-

Principle 5 — Infrastructure Supports Architecture

Infrastructure decisions shall serve the application architecture.

The infrastructure shall not force unnecessary complexity into the Domain or Application layers.

Infrastructure Context Model



Infrastructure Goals

The Infrastructure Architecture aims to provide:

- reliable application operation,
- secure deployment,
- manageable environments,
- controlled system evolution,
- operational visibility.

Infrastructure Evolution Approach

Mosaica infrastructure shall evolve gradually.

Initial versions should prefer:

- simple deployment,
- low operational overhead,
- clear ownership.

Advanced infrastructure patterns shall only be introduced when justified by system needs.

Module Structure

08 Infrastructure Architecture

08.01 Infrastructure Principles

08.02 Deployment Architecture

08.03 Environment Design

08.04 External Services

08.05 Monitoring & Reliability

08.06 Infrastructure Architecture Review

Architectural Decisions Introduced

Decision 01

Infrastructure complexity shall grow according to actual product requirements.

Decision 02

Infrastructure decisions shall preserve application and domain simplicity.

Decision 03

Operational reliability is a core infrastructure responsibility.

Decision 04

Environment separation is required for controlled development and deployment.

08.01 — Infrastructure Principles

Purpose

The purpose of this section is to define the fundamental infrastructure principles that guide the design, operation and evolution of the Mosaica infrastructure.

These principles establish the foundation for:

- deployment decisions,
- operational processes,
- scalability planning,
- reliability management,
- infrastructure evolution.

All infrastructure decisions shall be evaluated according to these principles.

Infrastructure Principles

Principle 1 — Simplicity First

Definition

Infrastructure shall remain as simple as possible while satisfying current product requirements.

Unnecessary infrastructure complexity shall be avoided.

Application

Mosaica shall avoid introducing infrastructure components without a clear operational need.

Examples of unnecessary early complexity:

- excessive service separation,
 - premature distributed systems,
 - unnecessary orchestration layers.
-

Reason

A simpler infrastructure provides:

- easier maintenance,
 - faster development,
 - fewer operational risks.
-

Principle 2 — Infrastructure Should Support Product Needs

Definition

Infrastructure decisions shall be driven by application requirements rather than technology trends.

Application

Infrastructure choices shall consider:

- current user needs,
 - expected workload,
 - development capacity,
 - operational requirements.
-

Rule

Technology shall serve the product.

The product shall not become constrained by unnecessary infrastructure choices.

Principle 3 — Environment Separation

Definition

Development, testing and production environments shall remain logically separated.

Environment Model

Development

↓

Testing

↓

Production

Purpose

Environment separation provides:

- safer experimentation,
 - controlled testing,
 - reliable releases.
-

Principle 4 — Deployment Repeatability

Definition

Application deployments should be predictable and repeatable.

The same deployment process should produce consistent results.

Application

Deployment processes should minimize:

- manual configuration,
 - undocumented steps,
 - environment-specific differences.
-

Principle 5 — Scalability Without Premature Complexity

Definition

The infrastructure shall support future growth without introducing unnecessary complexity in early stages.

Application

Initial infrastructure should support:

- clear scaling paths,
 - modular growth,
 - future optimization.
-

Rule

Scale when required by real system demand.

Do not build enterprise-scale infrastructure before it is needed.

Principle 6 — Reliability By Design

Definition

Infrastructure shall support stable system operation.

Reliability Requirements

Infrastructure planning should consider:

- backups,
 - monitoring,
 - recovery procedures,
 - failure handling.
-

Principle 7 — Security-Aware Infrastructure

Definition

Infrastructure shall support the security architecture defined in Module 07.

Application

Infrastructure decisions shall preserve:

- secure communication,
 - controlled access,
 - protected data storage.
-

Principle 8 — Observable Systems

Definition

Important system behavior should be visible through appropriate monitoring and logging.

Purpose

Operational visibility enables:

- problem detection,
 - performance analysis,
 - reliability improvement.
-

Principle 9 — Cost Awareness

Definition

Infrastructure decisions shall consider operational cost.

Application

The architecture should balance:

- performance,
- reliability,
- scalability,
- financial sustainability.

Infrastructure Decision Model

Infrastructure Decision

↓

Does it solve a real requirement?

↓

Does it increase maintainability?

↓

Does it preserve simplicity?

↓

Approve Infrastructure Change

Infrastructure Evolution Model

Mosaica infrastructure shall evolve through stages.

Initial Stage

↓

Growing Stage

↓

Initial Stage

Focus:

- simplicity,
 - fast development,
 - low operational overhead.
-

Growing Stage

Focus:

- improved reliability,
 - automation,
 - better monitoring.
-

Scaled Stage

Focus:

- advanced optimization,
 - high availability,
 - performance improvements.
-

Architectural Decisions Confirmed

Decision 01

Infrastructure complexity shall be introduced only when justified.

Decision 02

Infrastructure decisions must support product requirements.

Decision 03

Development and production environments shall remain separated.

Decision 04

Deployment processes should become repeatable and reliable.

Decision 05

Scalability shall be planned without premature overengineering.

Decision 06

Infrastructure shall support security, reliability and observability.

08.02 — Deployment Architecture

Purpose

The purpose of this section is to define the deployment architecture of Mosaica.

This section establishes how application components are packaged, deployed and operated across different environments.

The Deployment Architecture defines:

- deployment structure,
- application component placement,
- release flow,
- deployment responsibilities.

The objective is to create a predictable and maintainable deployment process.

Deployment Architecture Principles

Principle 1 — Component Separation

Application components shall have clearly defined deployment responsibilities.

Each component shall be independently maintainable.

Principle 2 — Consistent Deployment Process

Deployment procedures shall be repeatable across environments.

A deployment process should produce predictable results.

Principle 3 — Environment Independence

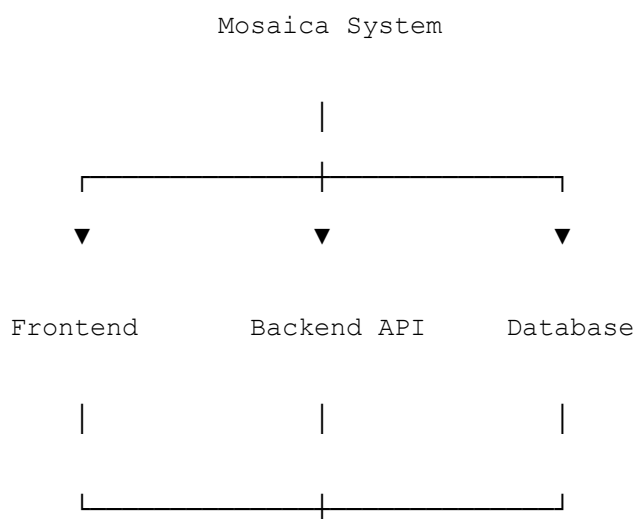
Application behavior shall not depend on undocumented environment-specific changes.

Principle 4 — Controlled Releases

Changes shall move through controlled deployment stages before reaching production.

Deployment Component Model

Mosaica deployment consists of the following primary components.



Frontend Deployment

Responsibility

Provides the user-facing application interface.

Responsibilities

Frontend deployment manages:

- user interface delivery,
 - client-side application execution,
 - communication with backend APIs.
-

Does Not Manage

Frontend does not:

- directly access the database,
 - contain business rules,
 - bypass API security.
-

Backend API Deployment

Responsibility

Provides the application communication layer.

Responsibilities

Backend deployment manages:

- API endpoints,
 - authentication flow,
 - Application Services,
 - workflow execution.
-

Architectural Position

Frontend

↓

Backend API

↓

Application Layer

↓

Domain Layer

Database Deployment

Responsibility

Provides persistent data storage.

Responsibilities

Database deployment manages:

- data persistence,
 - data availability,
 - backup support.
-

Does Not Manage

Database does not contain:

- business workflows,
- authorization logic,
- application decisions.

Deployment Flow

Mosaica follows a controlled deployment flow.

Development

↓

Testing

↓

Production

Development Deployment

Purpose

Used for active development and experimentation.

Characteristics

- frequent changes,
 - developer-focused configuration,
 - rapid iteration.
-

Testing Deployment

Purpose

Used for validation before release.

Characteristics

- controlled version,
- integration testing,
- quality verification.

Production Deployment

Purpose

Provides the live Mosaica application.

Characteristics

- stable releases,
 - monitoring enabled,
 - protected configuration.
-

Release Process

A release follows this sequence:

Code Change

↓

Build

↓

Testing

↓

Approval

↓

Deployment

↓

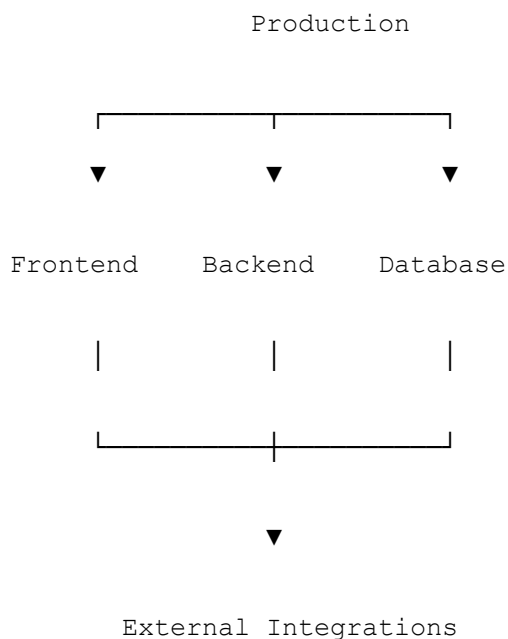
Monitoring

Deployment Responsibility Model

Component	Responsibility
Frontend	User interface delivery

Component	Responsibility
Backend API	Application communication
Application Layer	Workflow execution
Domain Layer	Business rules
Database	Data persistence
External Services	Supporting capabilities

Deployment Boundary Model



Deployment Evolution Strategy

Mosaica deployment shall evolve gradually.

Initial Stage

Focus:

- simple deployment,
- easy maintenance,
- fast iteration.

Growth Stage

Focus:

- automation,
 - improved reliability,
 - better deployment management.
-

Scale Stage

Focus:

- advanced availability,
 - performance optimization,
 - infrastructure automation.
-

Architectural Decisions Confirmed

Decision 01

Frontend, Backend and Database responsibilities remain separated.

Decision 02

The frontend does not directly communicate with persistence systems.

Decision 03

Backend deployment remains the entry point for application workflows.

Decision 04

Deployments follow controlled environment progression.

Decision 05

Infrastructure complexity grows according to actual system needs.

08.03 — Environment Design

Purpose

The purpose of this section is to define the environment architecture of Mosaica.

This section establishes how different application environments are structured and separated throughout the software lifecycle.

The objective is to provide a safe and predictable process for:

- development,
- testing,
- deployment,
- production operation.

Environment Design Principles

Principle 1 — Environment Separation

Development, testing and production environments shall remain logically separated.

Changes in one environment shall not unintentionally affect another environment.

Principle 2 — Environment Purpose Clarity

Each environment shall have a clearly defined purpose.

An environment shall not be used for responsibilities outside its intended role.

Principle 3 — Production Protection

Production environments shall receive additional protection because they contain real User data and live application functionality.

Principle 4 — Configuration Separation

Environment-specific configuration shall remain separate from application logic.

Environment Model

Mosaica uses three primary environments.

Environment Model

Development

↓

Testing

↓

Production

1. Development Environment

Purpose

The Development Environment is used for active software development.

Responsibilities

The Development Environment supports:

- feature development,
 - experimentation,
 - debugging,
 - local testing.
-

Characteristics

Development environment may contain:

- unfinished features,
 - test data,
 - frequent changes.
-

Restrictions

Development data shall not represent real User data.

2. Testing Environment

Purpose

The Testing Environment validates system behavior before production release.

Responsibilities

Testing Environment supports:

- integration testing,
 - workflow verification,
 - release validation.
-

Characteristics

The environment should resemble production behavior as closely as practical.

Restrictions

Testing activities shall not affect production data.

3. Production Environment

Purpose

The Production Environment provides the live Mosaica application.

Responsibilities

Production manages:

- real User access,
 - live application operations,
 - persistent User data.
-

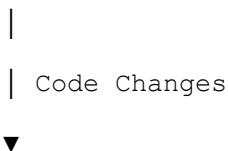
Characteristics

Production requires:

- controlled deployments,
 - monitoring,
 - backup support,
 - security protection.
-

Environment Relationship Model

Development



Testing



Production

Configuration Management

Each environment shall maintain separate configuration settings.

Examples:

Development

- Development database
- Debug settings

Testing

- Test database
- Validation settings

Production

- Production database
 - Secure settings
-

Data Separation

Development Data

Purpose:

Software development.

Characteristics:

- synthetic data,
 - temporary data,
 - safe experimentation.
-

Testing Data

Purpose:

Quality validation.

Characteristics:

- controlled test scenarios,
 - predictable datasets.
-

Production Data

Purpose:

Real application usage.

Characteristics:

- protected User data,
 - strict access control.
-

Deployment Promotion Flow

A release moves through environments in order.

Developer Change

↓

Development Validation

↓

Testing Validation

↓

Production Release

Environment Access Model

Access permissions shall differ by environment.

Development Access

Allows:

- developers,
- active experimentation.

Testing Access

Allows:

- developers,
- testers,
- validation processes.

Production Access

Restricted to authorized personnel only.

Environment Failure Isolation

A failure in one environment should not affect other environments.

Example:

Development Failure

X

Production
Unaffected

Environment Evolution

The environment structure may expand in the future if required.

Possible future additions:

- staging environment,
- preview environment,
- performance testing environment.

These shall only be introduced when justified.

Architectural Decisions Confirmed

Decision 01

Development, Testing and Production environments are separated.

Decision 02

Production environment receives the highest protection level.

Decision 03

Environment configuration remains separate from application code.

Decision 04

Production data shall never be used casually in development environments.

Decision 05

Additional environments require a clear operational need.

08.04 — External Services

Purpose

The purpose of this section is to define the architectural approach for external service integrations within Mosaica.

This section establishes how Mosaica communicates with external systems while maintaining:

- security,
- reliability,
- maintainability,
- architectural independence.

The objective is to ensure that external dependencies do not compromise the core application architecture.

External Service Principles

Principle 1 — Controlled External Dependency

Definition

External services shall be introduced only when they provide clear business or operational value.

Application

Mosaica shall avoid unnecessary external dependencies.

Every external integration should answer:

- What problem does it solve?
 - What value does it provide?
 - What is the impact if it fails?
-

Principle 2 — External Services Are Replaceable

Definition

External services shall not become tightly coupled with the core architecture.

Application

The application should communicate through defined interfaces rather than directly depending on external implementations.

Example

Incorrect:

Application Logic

↓

Specific External Service

Correct:

Application Logic

↓

External Service Interface

↓

External Provider

Principle 3 — Failure Isolation

Definition

Failure of an external service should not unnecessarily compromise the entire system.

Application

External dependencies should have:

- timeout handling,
 - failure responses,
 - fallback strategies when appropriate.
-

Principle 4 — Security-Aware Integration

Definition

External services shall follow Mosaica's security principles.

Requirements

External integrations shall protect:

- authentication information,
- User data,
- system credentials.

Principle 5 — Minimal Data Sharing

Definition

Mosaica shall share only the minimum required information with external services.

Application

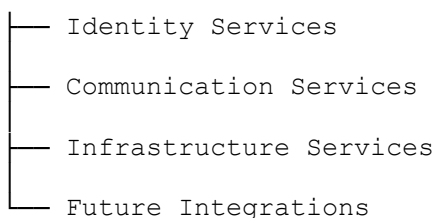
External systems shall not receive unnecessary:

- User information,
- personal archive data,
- internal system details.

External Service Categories

Mosaica external integrations are grouped into categories.

External Services



1. Identity Services

Purpose

Supports user authentication and identity-related operations.

Responsibilities

May provide:

- identity verification,
 - authentication support,
 - account management capabilities.
-

Architectural Rule

Identity services must not own Mosaica domain data.

2. Communication Services

Purpose

Supports communication features.

Examples

Potential future uses:

- email notifications,
 - user messages,
 - system announcements.
-

Architectural Rule

Communication services support workflows but do not define business rules.

3. Infrastructure Services

Purpose

Supports operational capabilities.

Examples

Potential future uses:

- file storage,
 - monitoring platforms,
 - logging systems.
-

Architectural Rule

Infrastructure services remain replaceable.

4. Future Integrations

Future features may introduce additional external dependencies.

Examples:

- external content providers,
 - recommendation services,
 - social integrations.
-

Requirement

Each future integration requires architectural evaluation before implementation.

External Service Communication Model

Mosaica Application

↓

External Service Interface

↓

External Provider

External Service Failure Handling

External failures shall be handled through controlled application behavior.

Example:

External Service Failure

↓

Failure Detection

↓

Controlled Response

↓

User Experience Preservation

External Service Security Model

External System

↓

Secure Communication

↓

Integration Boundary

↓

Mosaica Application

Dependency Evaluation Criteria

Before introducing an external service, evaluate:

Business Value

Does it solve a real product requirement?

Reliability

Can the system handle service interruptions?

Security

Does it preserve data protection requirements?

Maintainability

Can the dependency be replaced if necessary?

Cost

Is the operational cost justified?

Architectural Decisions Confirmed

Decision 01

External services require clear architectural justification.

Decision 02

External dependencies shall remain replaceable.

Decision 03

External failures shall be isolated from core application behavior.

Decision 04

External services shall not contain Mosaica business rules.

Decision 05

Future integrations require architectural evaluation.

08.05 - Monitoring & Reliability

Purpose

The purpose of this section is to define the monitoring and reliability architecture of Mosaica.

This section establishes how the system observes, maintains and improves operational stability.

The objective is to ensure that Mosaica can:

- detect problems,
 - preserve data integrity,
 - recover from failures,
 - maintain reliable operation.
-

Monitoring & Reliability Principles

Principle 1 — Operational Visibility

Definition

Important system behavior should be observable.

The system should provide enough information to understand its operational state.

Application

Monitoring should consider:

- application health,
 - errors,
 - performance issues,
 - system availability.
-

Principle 2 — Appropriate Complexity

Definition

Monitoring solutions shall match the current system scale.

Application

Personal usage phase:

- simple logging,
- basic health checks,
- manual review when necessary.

Commercial phase:

- advanced monitoring,
 - automated alerts,
 - operational dashboards.
-

Principle 3 — Failure Detection

Definition

The system should be capable of detecting important failures.

Examples

Possible failures:

- application errors,
 - database connection problems,
 - external service failures.
-

Principle 4 — Data Reliability

Definition

Mosaica shall prioritize preservation of User data.

Application

Reliability planning shall consider:

- backups,
 - recovery processes,
 - data integrity checks.
-

Principle 5 — Graceful Failure

Definition

When failures occur, the system should fail in a controlled manner.

Application

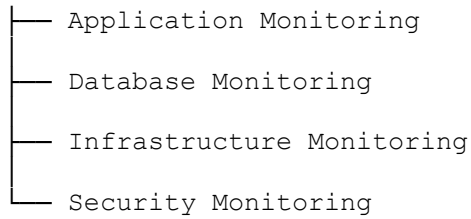
The system should:

- protect existing data,
 - avoid invalid states,
 - provide understandable error responses.
-

Monitoring Areas

Mosaica monitoring is divided into four primary areas.

Monitoring



1. Application Monitoring

Responsibility

Observes application behavior.

Monitored Areas

Examples:

- application errors,
 - failed operations,
 - response performance,
 - workflow failures.
-

Purpose

Identify problems affecting User experience.

2. Database Monitoring

Responsibility

Observes data storage health.

Monitored Areas

Examples:

- database availability,
 - storage usage,
 - backup status,
 - data integrity.
-

Purpose

Protect persistent User information.

3. Infrastructure Monitoring

Responsibility

Observes system operation environment.

Monitored Areas

Examples:

- service availability,
 - resource usage,
 - deployment status.
-

Purpose

Maintain reliable system operation.

4. Security Monitoring

Responsibility

Observes security-related events.

Monitored Areas

Examples:

- authentication failures,
- suspicious access attempts,
- permission errors.

Logging Architecture

Mosaica uses structured logging principles.

Application Event

↓

Logging System

↓

Analysis / Review

Logging Principles

Principle 1

Logs should support troubleshooting.

Principle 2

Logs shall not expose sensitive User information.

Principle 3

Important system events should be traceable.

Reliability Model

Mosaica reliability is based on three capabilities.

Reliability

└─ Prevention

└─ Detection

└─ Recovery

Prevention

Purpose:

Reduce failures before they happen.

Examples:

- validation,
 - secure workflows,
 - controlled deployments.
-

Detection

Purpose:

Identify problems quickly.

Examples:

- error logging,
 - health checks,
 - monitoring.
-

Recovery

Purpose:

Restore normal operation.

Examples:

- backups,
- data restoration,
- rollback procedures.

Backup Strategy

Principle

User-owned data requires protection against accidental loss.

Backup Considerations

Future implementations should consider:

- database backups,
- export options,
- recovery procedures.

Recovery Model

In case of failure:

Failure

↓

Detection

↓

Assessment

↓

Recovery Action

↓

System Restoration

Reliability Evolution

Mosaica reliability requirements evolve with system growth.

Personal Usage

↓

Small User Base

↓

Public Product

Personal Usage Phase

Focus:

- simple backups,
 - basic logs,
 - manual recovery.
-

Product Phase

Focus:

- automated monitoring,
 - alerting,
 - high availability,
 - disaster recovery.
-

Architectural Decisions Confirmed

Decision 01

Monitoring complexity shall match system maturity.

Decision 02

Data reliability is a primary infrastructure responsibility.

Decision 03

Logging shall support troubleshooting without exposing sensitive data.

Decision 04

The system shall support failure detection and recovery processes.

Decision 05

Advanced reliability features shall be introduced only when justified.

08.06 — Infrastructure Architecture Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica Infrastructure Architecture provides a reliable and maintainable operational foundation while preserving the principles established in previous architectural modules.

This review confirms that infrastructure decisions support:

- application operation,
 - security requirements,
 - scalability needs,
 - operational simplicity,
 - future evolution.
-

Scope Reviewed

The following infrastructure architecture components have been reviewed:

- Infrastructure Principles
- Deployment Architecture
- Environment Design
- External Services

- Monitoring & Reliability
-

Validation Results

Infrastructure Alignment

Status: Approved

The Infrastructure Architecture is consistent with the approved:

- Application Architecture,
- Security Architecture,
- API Architecture.

Infrastructure decisions support the application structure without introducing unnecessary coupling.

Simplicity and Evolution Strategy

Status: Approved

The infrastructure approach correctly follows the principle:

Infrastructure complexity shall grow according to actual system requirements.

The architecture avoids premature introduction of:

- complex deployment systems,
 - unnecessary infrastructure layers,
 - excessive operational overhead.
-

Personal Usage Compatibility

Status: Approved

The infrastructure architecture supports the initial personal usage phase of Mosaica.

Confirmed:

- Single-user operation does not require mandatory paid infrastructure.
- Local execution remains a valid architectural option.
- Future commercial infrastructure can be introduced when justified.

Deployment Architecture

Status: Approved

Deployment responsibilities are correctly separated.

Confirmed:

Frontend

↓

Backend API

↓

Application Layer

↓

Domain Layer

↓

Database

The database is not directly exposed to external clients.

Environment Separation

Status: Approved

The environment strategy provides controlled development and deployment flow.

Confirmed environments:

Development

↓

Testing

↓

Production

Each environment has a distinct purpose.

External Service Management

Status: Approved

External dependencies are treated as replaceable components.

Confirmed principles:

- External services do not contain Mosaica business rules.
 - External failures should be isolated.
 - New integrations require architectural evaluation.
-

Monitoring and Reliability

Status: Approved

The reliability approach is appropriate for the current product maturity.

Confirmed:

- Error visibility,
- data protection,
- recovery considerations,
- operational awareness.

Advanced monitoring systems remain future considerations.

Infrastructure Cost Strategy

Status: Approved

The infrastructure strategy recognizes different product phases.

Personal Usage

↓

Optional Online Usage

↓

Commercial Product

Each phase may require different infrastructure investment.

Paid infrastructure shall be introduced only when justified by:

- user growth,
 - product requirements,
 - operational needs.
-

Architectural Decisions Confirmed

Decision 01

Infrastructure shall remain simple during early product phases.

Decision 02

Single-user operation shall remain possible without mandatory infrastructure costs.

Decision 03

Infrastructure shall evolve according to validated requirements.

Decision 04

Deployment, environment and operational responsibilities remain separated.

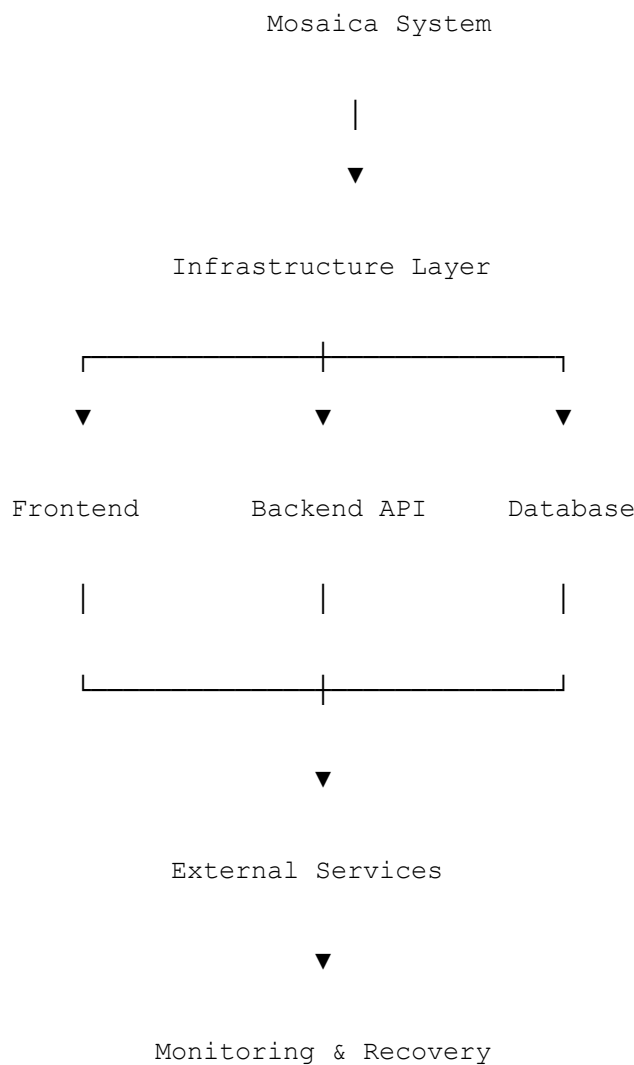
Decision 05

External dependencies shall remain replaceable.

Decision 06

Reliability requirements shall increase proportionally with system maturity.

Infrastructure Architecture Summary



Review Conclusion

The Mosaica Infrastructure Architecture is considered architecturally complete.

The current design provides:

- a maintainable operational foundation,
- support for personal usage,
- a clear migration path toward future product deployment,
- compatibility with security and application requirements.

No blocking architectural issues have been identified.

ADR — Personal Use Infrastructure Strategy

Status

Accepted

Context

Mosaica was initially designed as a personal digital archive system.

During the early development phase, the primary goal is to create and use the system personally before considering any commercial deployment.

At this stage:

- The number of users is limited to one.
- The system does not require production-scale infrastructure.
- Operational costs should be minimized.
- Development speed and simplicity are higher priorities than large-scale infrastructure.

Introducing paid infrastructure services before a validated need exists would create unnecessary cost and complexity.

Decision

Mosaica shall support a zero-cost personal usage phase before any commercial deployment.

The initial architecture shall allow the system to operate without mandatory paid services for a single-user environment.

The personal usage phase should prioritize:

- local execution when appropriate,
 - free-tier compatible services when external access is required,
 - simple and maintainable infrastructure choices.
-

Consequences

Positive Consequences

- The project can be developed and used without financial commitment.
 - Infrastructure complexity remains low.
 - Development focuses on product value rather than operational costs.
 - Future migration to commercial infrastructure remains possible.
-

Negative Consequences

- Personal-use infrastructure may not immediately support large numbers of users.
 - Some commercial-scale features may require future architectural expansion.
-

Future Transition

If Mosaica evolves into a public product, infrastructure decisions may be revisited.

Future commercial deployment may introduce:

- hosted databases,
- cloud infrastructure,
- paid services,
- advanced monitoring systems.

Such changes shall be evaluated according to actual product requirements.

Related Principles

This decision supports the following Mosaica principles:

- Simplicity First
 - Scalability Without Premature Complexity
 - Infrastructure Should Support Product Needs
 - Security By Design
-

Decision Summary

Mosaica shall remain usable as a personal system without requiring paid infrastructure until real product requirements justify additional investment.